

Suffix Trees

1 Motivation

Suffix Tree

A **suffix tree** is a compressed trie built from all suffixes of one text. It is used for fast pattern searching inside that text.

A normal trie stores a dictionary of full strings. A suffix tree stores every suffix of a single text, so any substring search can be converted into a path search.

Substring Rule

A pattern P occurs in T exactly when the characters of P can be traced from the root of the suffix tree. The leaves below the matched point give the starting positions.

2 Suffixes

For a text

$$T = t_1 t_2 \cdots t_m,$$

the suffixes are

$$T[1..m], \quad T[2..m], \quad \dots, \quad T[m..m].$$

Example

For

$$T = \text{banana},$$

some suffixes are

banana, anana, nana, ana, na, a.

3 Terminal Symbol

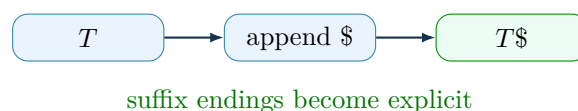
End Marker

A unique terminal symbol $\$$ is often appended to the text:

$$T\$.$$

The symbol $\$$ does not occur elsewhere in the alphabet.

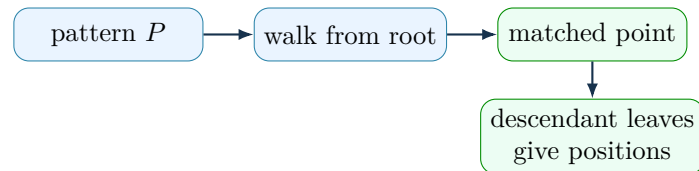
The terminal symbol ensures no suffix is accidentally treated as a prefix of another suffix.



4 Purpose

To search for a pattern P , follow the characters of P from the root across edge labels.

- If the complete pattern can be followed, then P occurs in T .
- Leaves below the matched point store starting positions of occurrences.



5 Complexity

For a text of length m , the recorded suffix-tree bounds are:

$$\boxed{\text{construction time} = \mathcal{O}(m)}$$

and

$$\boxed{\text{space} = \mathcal{O}(m)}.$$

Without compression, storing all suffixes explicitly would take

$$1 + 2 + \dots + m = \frac{m(m+1)}{2} = \mathcal{O}(m^2)$$

characters. A compact suffix tree stores the same suffix information using only $\mathcal{O}(m)$ tree space.